

03

 UNIDAD I · FUNDAMENTOS DE SISTEMAS DISTRIBUIDOS CONFIABLES

FIRMAS DIGITALES Y ANATOMÍA DE LA CADENA

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

180 MIN

DURACIÓN

**TEÓRICO-
PRÁCTICA**

MODALIDAD

**LAB 01
ENTREGADO**

PRERREQUISITO

**MINI-
BLOCKCHAIN**

ENTREGABLE



LO QUE HAY QUE TRAER

01

EL LABORATORIO 01 FUNCIONANDO

El de hoy REUTILIZA la función de hash y el árbol de Merkle de la sesión pasada. Se entrega una implementación de referencia, pero conviene llegar con la propia.

02

LA DEPENDENCIA NUEVA INSTALADA

Está en el archivo de requisitos del laboratorio. Instalarla antes de clase ahorra diez minutos.

03

LA PREGUNTA DE LA SEMANA PENSADA

Qué le falta a un hash para demostrar además quién escribió el dato. Abre la sesión.



DÓNDE ESTAMOS

Esta sesión cierra la Unidad I: después de hoy, todo lo que la Sesión 1 afirmó queda demostrado y programado.



MAPA DE LA SESIÓN

MIN	BLOQUE	CONTENIDO
00–10	Apertura	Plenaria de la pregunta de la semana y revisión del entregable de la Sesión 2.
10–25	A.1	El problema de la autoría: por qué un hash no basta y por qué fallan los intentos ingenuos.
25–42	A.2	Criptografía asimétrica: la función de un solo sentido con trampa y la curva secp256k1.
42–58	A.3	ECDSA: firmar y verificar. Qué prueba una firma y qué no. El fallo que revela la clave.
58–70	A.4	De clave pública a dirección: keccak-256, los últimos 20 bytes y el checksum EIP-55.
70–78	Pausa	—
78–92	B.1	Anatomía del bloque: cabecera y cuerpo, campo por campo.
92–105	B.2	Encadenamiento e inmutabilidad, ahora con todas las piezas en la mano.
105–118	B.3	La red P2P: propagación, mempool y el conflicto que queda abierto.
118–170	BLOQUE C	Laboratorio 02: firmas, dirección y mini-blockchain con transacciones firmadas.
170–180	BLOQUE D	Cierre, entregable y puente hacia el consenso.



MIN 10 – 70 · EXPOSICIÓN DIALOGADA

CRIPTOGRAFÍA ASIMÉTRICA

Cómo se demuestra la autoría ante alguien que no te conoce, sin revelarle ningún secreto.



LA SESIÓN 2 RESOLVIÓ MEDIA PREGUNTA

Sabemos comprobar que un dato no cambió: se publica su hash y cualquiera verifica. Pero eso deja abierta la otra mitad, que es la que hace falta para que exista el dinero digital.

Un hash lo puede calcular cualquiera. ¿Cómo demuestro que FUI YO quien escribió el dato, ante alguien que no me conoce y sin revelarle ningún secreto?

Antes de dar la respuesta conviene agotar los caminos que no funcionan, porque cada fracaso deja claro un requisito.



POR QUÉ LA CLAVE COMPARTIDA FRACASA

INTENTO 1 · CONTRASEÑA COMPARTIDA

Ana y Bruno acuerdan un secreto. Ana publica el mensaje junto a hash(secreto + mensaje). Bruno recalcula y comprueba. Funciona — es una construcción real —, pero tiene dos fallas que lo descartan.

NO ESCALA — hace falta un secreto distinto con cada persona, y distribuirlo por un canal seguro que todavía no existe.

EL VERIFICADOR PUEDE SUPLANTAR — Bruno conoce el secreto, así que Bruno puede fabricar mensajes que parezcan de Ana.

INTENTO 2 · PUBLICAR EL SECRETO PARA QUE TODOS VERIFIQUEN

Y ahora cualquiera la fabrica. El sistema no distingue a Ana de un impostor. Se derrumba de inmediato.

El requisito: que quien puede VERIFICAR una firma no pueda PRODUCIRLA.



AUTENTICIDAD, INTEGRIDAD Y NO REPUDIO

PROPIEDAD	QUÉ SIGNIFICA
Autenticidad	El mensaje proviene de quien controla una clave concreta. Nadie más pudo producir esa firma.
Integridad	El mensaje no fue alterado después de firmarse. Cambiar un byte invalida la firma.
No repudio	Quien firmó no puede negar después haberlo hecho, porque nadie más tenía la capacidad de hacerlo.



EL NO REPUDIO ES TÉCNICO, NO JURÍDICO

La firma demuestra que SE USÓ UNA CLAVE CONCRETA. No demuestra que la persona a la que atribuimos esa clave estuviera consciente, de acuerdo, o siquiera presente.

Si roban una clave privada, el sistema seguirá considerando auténtica cada firma del ladrón — y matemáticamente tendrá razón. Es lo que ocurre en cada robo de billetera: no falla la criptografía, falla la custodia.



UNA FUNCIÓN DE UN SOLO SENTIDO, CON TRAMPA

Ya conocemos una función de un solo sentido: el hash de la Sesión 2. Pero al hash le falta algo — es DEMASIADO irreversible, no deja ninguna puerta. Lo que hace falta es una que sea irreversible para todo el mundo salvo para quien conoce un dato secreto.



ANALOGÍA · EL CANDADO ABIERTO

Imagine que reparte por la ciudad candados abiertos con su nombre grabado, y se queda con la única llave. Cualquiera puede cerrar uno de sus candados; solo usted puede abrirlo.

Para la FIRMA hay que darle la vuelta: usted usa la llave para producir algo que solo su llave puede producir, y cualquiera comprueba con el candado que salió de allí. La operación privada produce; la pública verifica.

OPERACIÓN	COSTE
Dados k y G , calcular $P = k \cdot G$	BARATO. Hay atajos que lo resuelven en unos cientos de operaciones aunque k tenga 256 bits.
Dados P y G , recuperar k	INVIABLE. No se conoce método sustancialmente mejor que probar valores. Con 256 bits está fuera del alcance de cualquier cómputo concebible.



SECP256K1 Y EL PAR DE CLAVES



FICHA DE TÉRMINO 02 · LA CURVA DE BITCOIN Y ETHEREUM

SECP256K1: curva elíptica definida por $y^2 = x^3 + 7$ sobre un cuerpo finito. Trae fijados un punto generador G y un orden n , que es la cantidad de puntos distintos que se pueden alcanzar.

CLAVE PRIVADA

Un número entero aleatorio entre 1 y $n-1$. En la práctica, 32 bytes.
No se «genera» con un algoritmo elaborado: se SORTEA.

CLAVE PÚBLICA

El punto $P = k \cdot G$. Son dos coordenadas de 32 bytes cada una: 64 bytes en total. Publicarla no compromete nada.

El nombre no es decorativo: sec por el consorcio que la estandarizó, p256 por los bits, k por Koblitz —la familia de curvas— y 1 por ser la primera de esa lista.



LA SEGURIDAD ESTÁ EN EL SORTEO, NO EN LA CURVA



ALERTA PROFESIONAL

Si la clave privada es simplemente un número aleatorio, toda la seguridad del sistema descansa en la calidad de ese azar. Una clave generada con un generador predecible es tan insegura como la contraseña «1234», por impecable que sea la curva.

No es teórico. Ha habido incidentes reales, con pérdidas grandes, causados exclusivamente por generadores de aleatoriedad defectuosos en dispositivos y bibliotecas — no por debilidades de la criptografía.

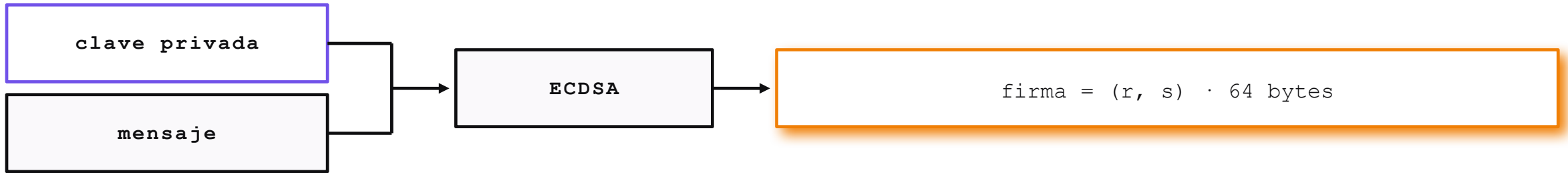
Nunca se genera una clave con la función de azar de propósito general del lenguaje. Se usa la fuente criptográficamente segura del sistema.

Es lo que hacen por defecto las bibliotecas serias — y es lo que se usará en el laboratorio.



QUIEN VERIFICA NUNCA TOCA LA CLAVE PRIVADA

FIRMAR · OPERACIÓN PRIVADA



VERIFICAR · OPERACIÓN PÚBLICA

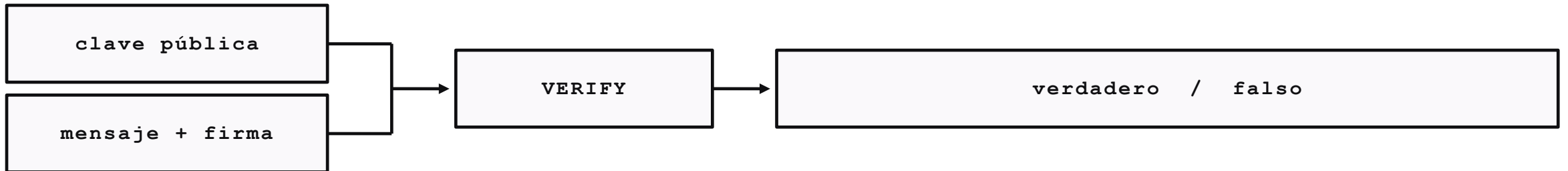


Fig. 1 · toda la asimetría cabe en esta diferencia



UNA FIRMA, DE VERDAD

```
// NUNCA usar esta clave con fondos reales: es material de clase.  
  
clave privada 7fa03ea163b89b38dfe61c7070b0894b2b5b3eee14187af429a90d17ec19fd1e  
               32 bytes · el secreto absoluto  
  
clave pública edafd5fc366ffb944d605a7050235b9d ... 9f0aaadac58bb61d  
               64 bytes · coordenada X seguida de coordenada Y  
  
mensaje      "Transfiero 10 lotes a Bruno"  
  
firma        c877ef924d6ed057e27af4e21bb6a36f8ce97565a85d5172535f8a541aec3d30  
              f21fe2d75a6cab0e93b080a2020b02cebfc8d5ed6990d90fcde7a432966fe023
```

`verificar(mensaje, firma, pública) → VERDADERO`

`verificar("Transfiero 100 lotes...", firma, pública) → FALSO`



QUÉ PRUEBA UNA FIRMA Y QUÉ NO PRUEBA

SÍ PRUEBA	NO PRUEBA
Que quien produjo la firma tenía la clave privada correspondiente.	Quién es esa persona. La clave no lleva nombre, cédula ni nacionalidad.
Que el mensaje no cambió ni un byte después de firmarse.	Que el firmante entendiera lo que firmaba. Es la base de casi todos los robos a usuarios.
Que la firma corresponde a ese mensaje y no a otro.	Cuándo se firmó. No hay marca temporal dentro de la firma.
Que nadie sin la clave pudo fabricarla.	Que el firmante fuera el dueño legítimo de la clave, y no alguien que se la robó.



LO QUE HAY QUE RETENER

Una firma prueba que se usó una clave. Todo lo demás son supuestos que hay que sostener por otros medios.

EL NÚMERO QUE NO SE PUEDE REPETIR

REPETIRLO REVELA LA CLAVE PRIVADA

Cada firma ECDSA necesita internamente un número aleatorio de un solo uso. Si ese número SE REPITE en dos firmas hechas con la misma clave, cualquiera que vea ambas puede despejar la clave privada con álgebra elemental. No hace falta romper nada: basta resolver un sistema de dos ecuaciones.

Ha ocurrido en producción más de una vez, con consecuencias graves: consolas cuya clave de firmado quedó expuesta, y billeteras móviles vaciadas porque el generador del sistema devolvía valores repetidos.

LA DEFENSA ESTÁNDAR

Derivar ese número de forma determinista a partir de la clave privada y del mensaje, en lugar de sortearlo. Así nunca se repite mientras el mensaje sea distinto, y no depende del azar del dispositivo.

No se implementa criptografía a mano. Se usan bibliotecas establecidas y se comprueban contra vectores de prueba conocidos.

LA DIRECCIÓN ES LA COLA DEL HASH DE LA CLAVE PÚBLICA

1 · CLAVE PRIVADA · 32 bytes · sorteada al azar

7fa03ea163b89b38dfe61c7070b0894b2b5b3eee14187af429a90d17ec19fd1e

multiplicación sobre la curva · $k \cdot G$

2 · CLAVE PÚBLICA · 64 bytes · coordenada X seguida de Y

edafd5fc366ffb944d605a7050235b9d ... 9f0aaadac58bb61d

keccak-256

3 · HASH DE LA CLAVE PÚBLICA · 32 bytes

3c966aba52e0ac36472673387**32805fbe544f6117b755512f2029e3e8b6aa135**

4 · DIRECCIÓN = LOS ÚLTIMOS 20 BYTES · se descartan los 12 primeros



EL CHECKSUM DE EIP-55



UNA SUMA DE VERIFICACIÓN QUE NO OCUPA UN SOLO BYTE

Se toma la dirección en minúsculas y se calcula su hash. Después, para cada carácter: si es una LETRA y el dígito correspondiente del hash es 8 o mayor, esa letra se escribe en mayúscula. Los dígitos numéricos no cambian, porque no tienen mayúscula.

```
sin checksum    0x732805fbe544f6117b755512f2029e3e8b6aa135  
con EIP-55      0x732805FbE544F6117B755512f2029e3e8B6aa135
```

Son la MISMA dirección: para la red, las mayúsculas son irrelevantes. Pero un programa que valide el patrón detecta casi cualquier error de transcripción antes de enviar los fondos.



LO QUE EL CHECKSUM NO HACE

Detecta errores de transcripción, no direcciones inexistentes. Enviar fondos a una dirección bien formada pero sin dueño los destruye para siempre.

A large, bold, white capital letter 'B' is centered within a solid orange square.

MIN 78 — 118 · EXPOSICIÓN DIALOGADA

ANATOMÍA DE LA CADENA

Ya están todas las piezas. Un bloque es la estructura que las ensambla.



LA CABECERA DEL BLOQUE, CAMPO POR CAMPO

CAMPO DE LA CABECERA	PARA QUÉ SIRVE	DE DÓNDE SALE
Hash del bloque anterior	El eslabón. Es lo que convierte una lista de bloques en una cadena.	Se copia del bloque padre.
Raíz de Merkle	Compromete el contenido completo del cuerpo en 32 bytes.	SESIÓN 2. Se calcula sobre las transacciones del bloque.
Marca temporal	Cuándo declara el proponente que se creó el bloque.	La pone quien propone, dentro de márgenes que la red valida.
Dificultad	Cuán costoso debía ser el trabajo para que el bloque sea válido.	La calcula el protocolo, no el minero. Sesión 4.
Nonce	El campo que se recorre por fuerza bruta hasta cumplir la dificultad.	Lo busca quien mina. Sesión 4.



LA OBSERVACIÓN QUE HAY QUE HACER

El cuerpo puede pesar megabytes, pero solo se hasha la cabecera. Y aun así el hash del bloque queda comprometido con todas sus transacciones — porque la raíz de Merkle está DENTRO de la cabecera.



LA DIVISIÓN DE RESPONSABILIDADES

Las firmas no están en la cabecera: viajan dentro de cada transacción, en el cuerpo. Cada nodo las verifica de forma independiente antes de aceptarlas.

LA FIRMA

Sesión 3

¿Esta transacción la autorizó quien controla esos fondos?

EL ÁRBOL DE MERKLE

Sesión 2

¿Esta transacción está en este bloque?

EL ENCADENAMIENTO POR HASH

Sesión 3

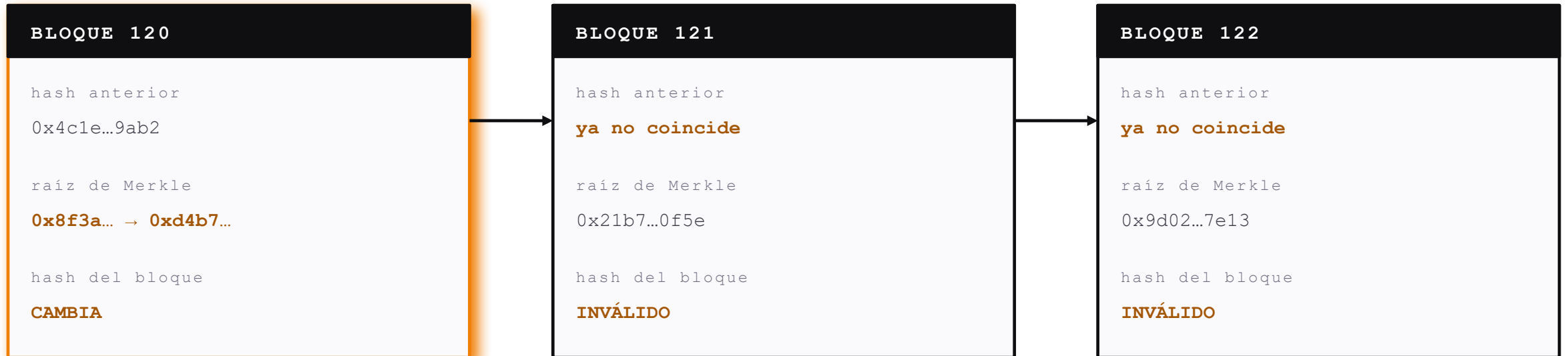
¿Este bloque sigue siendo el que era?

EL CONSENSO

Sesión 4

¿Es esta la cadena en la que todos estamos de acuerdo?

EL EFECTO CASCADA, CON LA PIEZA QUE LO EXPLICA



SE ALTERA UNA TRANSACCIÓN DEL BLOQUE 120

cambia su raíz de Merkle · cambia el hash de la cabecera · el bloque siguiente ya no apunta a un hash que exista · y así hasta el final de la cadena



LO QUE EL ENCADENAMIENTO NO RESUELVE

FICHA DE TÉRMINO 05 · INMUTABILIDAD, CON PRECISIÓN

Alterar un registro escrito es económicamente inviable, no físicamente imposible. Habría que rehacer el trabajo de todos los bloques posteriores más rápido de lo que la red produce bloques nuevos. De ahí la práctica de esperar CONFIRMACIONES: cada bloque encima multiplica el costo de revertir.

EL HUECO QUE QUEDA

El encadenamiento garantiza que UNA cadena dada es internamente coherente. No dice nada sobre cuál de dos cadenas coherentes es la buena.

Un atacante puede construir su propia cadena, perfectamente encadenada, con las transacciones que le convengan. Todas sus firmas serán válidas y todos sus hashes cuadrarán. El encadenamiento no lo detecta, porque no hay nada que detectar: esa cadena es válida.



PROPAGACIÓN POR RUMOR Y MEMPOOL

No hay servidor ni directorio central. Cada nodo mantiene conexiones con un puñado de vecinos y aplica una regla simple: cuando recibo algo válido que no conocía, lo valido y se lo paso a mis vecinos.

FICHA DE TÉRMINO 06 · MEMPOOL

Conjunto de transacciones que un nodo ha recibido y considerado válidas, pero que todavía no están en ningún bloque. Es la sala de espera: cuando en la demostración de la Sesión 2 la transacción apareció como «pendiente», estaba aquí.

CADA NODO TIENE EL SUYO

y no son idénticos: dependen de qué le llegó a cada uno y en qué orden.

ES PÚBLICO

cualquiera observa las transacciones pendientes antes de que se ejecuten. De ahí el reordenamiento de la Sesión 9.

ESTAR AHÍ NO GARANTIZA NADA

una transacción puede quedarse indefinidamente si nadie tiene incentivo para incluirla.

EL CONFLICTO QUE QUEDA ABIERTO

Dos nodos producen un bloque válido casi al mismo tiempo, apuntando al mismo padre. Ambos son correctos. Media red ve uno y media red ve el otro. Nada de lo construido hasta hoy resuelve esto.

C



MIN 118 – 170 · EN MÁQUINA, EN PAREJAS

LABORATORIO 02

Doscientas líneas que reúnen toda la Unidad I: firmas, Merkle y encadenamiento. Y después romperlo.



CLAVES Y FIRMAS

01

Generar un par de claves

`generar_par()` usando la fuente de aleatoriedad segura de la biblioteca. Observar los tamaños: 32 bytes la privada, 64 la pública.

02

Firmar y verificar

`firmar(privada, mensaje)` y `verificar publica, mensaje, firma`). Comprobar que una firma válida verifica.

03

Romperla de tres maneras

Alterar el mensaje, alterar un byte de la firma, y usar la clave pública de otro par. LAS TRES DEBEN FALLAR. Una función que solo acepta lo correcto está a medio escribir.



DIRECCIÓN Y CHECKSUM

01

Derivar la dirección

keccak-256 de los 64 bytes de la clave pública, quedarse con los ÚLTIMOS 20. Contrastar con el valor de referencia del archivo de pruebas.

02

Aplicar EIP-55

Hashear la dirección en minúsculas y poner en mayúscula cada letra cuyo dígito correspondiente del hash sea 8 o mayor.

03

Detectar un error de transcripción

eip55_valido() debe rechazar una dirección a la que se le cambió una sola letra de caja.



LA MINI-BLOCKCHAIN

01

Transacciones firmadas

Origen, destino, monto, contador y firma. Serialización DETERMINISTA —siempre los mismos campos en el mismo orden— y firma sobre esa serialización. Si dos nodos serializan distinto, la firma no verifica.

02

El bloque

Cabecera con hash del padre, raíz de Merkle de sus transacciones —REUTILIZANDO EL LABORATORIO 01—, marca temporal y nonce. El hash se calcula solo sobre la cabecera.

03

Validación de la cadena

`cadena_valida()` comprueba para cada bloque: que el hash del padre coincida con el hash real del anterior, que la raíz corresponda a sus transacciones, y que TODAS las firmas verifiquen.

04

Romper la cadena

Alterar el monto de una transacción de un bloque intermedio y validar. Observar QUÉ falla y DÓNDE: primero la firma, después la raíz de Merkle, después el eslabón de todos los posteriores.

EL PASO 04 ES LA COMPROBACIÓN CENTRAL Y EL CIERRE DE LA UNIDAD I: LA FIG. 3, EJECUTÁNDOSE.



ERRORES FRECUENTES DEL LABORATORIO

SÍNTOMA	CAUSA Y SOLUCIÓN
La firma verifica en una máquina y no en otra	Serialización no determinista: campos en distinto orden, o un diccionario sin orden fijo. Fija el orden y hashea bytes, no la representación de un objeto.
La dirección no coincide con la de referencia	Casi siempre se hashea la clave pública CON el prefijo de formato. Hay que hashear los 64 bytes de las coordenadas, sin byte inicial.
Se usó SHA-3 en lugar de Keccak	La trampa de la Sesión 2. Comprueba el vector conocido: keccak256("") empieza por c5d2460186f7233c.
La cadena valida aunque se alteró un bloque	La validación lee el hash guardado en el bloque en lugar de recalcularlo desde la cabecera. Es el error conceptual más grave de este laboratorio.
Todas las firmas fallan tras cambiar un campo	Correcto. Es el resultado esperado del paso 4, no un fallo.



D

MIN 170 – 180 · PLENARIA

CIERRE

Qué llevarse, qué entregar y el hueco que abre la Sesión 4.

LO QUE HAY QUE LLEVARSE DE ESTA SESIÓN

UNO

El requisito era ASIMETRÍA: que quien puede verificar no pueda firmar. Los esquemas de clave compartida fracasan por eso.

DOS

La clave privada es UN NÚMERO SORTEADO AL AZAR. Toda la seguridad descansa en la calidad de ese sorteo.

TRES

Una firma prueba QUE SE USÓ UNA CLAVE. No prueba identidad, ni intención, ni momento, ni comprensión.

CUATRO

La dirección es literalmente LA COLA DEL HASH de la clave pública, y su patrón de mayúsculas es un checksum gratuito.

CINCO

El bloque ensambla las tres piezas: firmas en el cuerpo, raíz de Merkle en la cabecera, hash del padre como eslabón.

SEIS

Nada de lo construido resuelve CUÁL DE DOS CADENAS VÁLIDAS es la buena. Ese hueco tiene nombre y fecha: Sesión 4.

QUEDA UN HUECO PERFECTAMENTE DELIMITADO

Sabemos construir una cadena válida. No sabemos decidir cuál de varias cadenas válidas es LA cadena.

Pregunta de la semana: si un atacante construye su propia cadena, perfectamente encadenada y con todas las firmas válidas, ¿qué impide que la red la acepte? Proponga un criterio y busque cómo se lo saltaría usted mismo.

SESIÓN 04 · CONSENSO, INCENTIVOS Y EL TRILEMA

Generales bizantinos, prueba de trabajo, prueba de participación, bifurcaciones y finalidad, y el trilema de escalabilidad. Cierra la Unidad I con el QUIZ 1, que evalúa todo lo visto desde la Sesión 1.



GLOSARIO DE LA SESIÓN

Solo los términos que introduce esta sesión. Los de las sesiones 1 y 2 siguen siendo exigibles.

Autenticidad		Garantía de que un mensaje proviene de quien controla una clave concreta.
Cabecera de bloque	<code>block header</code>	Parte pequeña y de tamaño fijo con el hash del padre, la raíz de Merkle, la marca temporal, la dificultad y el nonce. Es lo único que se hashea.
Clave privada		Número entero de 32 bytes sorteado al azar. Quien lo tiene controla los fondos. No se deriva de nada ni se recupera si se pierde.
Clave pública		Punto de la curva obtenido multiplicando el generador por la clave privada. Son 64 bytes. Publicarla no compromete nada.
ECDSA	<code>Elliptic Curve DSA</code>	Algoritmo de firma sobre curva elíptica. Produce un par (r, s) de 64 bytes a partir del hash del mensaje y la clave privada.
EIP-55		Convención que codifica una suma de verificación en el patrón de mayúsculas de una dirección, sin ocupar espacio adicional.

GLOSARIO · CONTINUACIÓN

Logaritmo discreto elíptico		Recuperar k dados $P = k \cdot G$ y G . Calcular P es barato; recuperar k es inviable. Esa asimetría sostiene toda la criptografía de clave pública del curso.
Mempool	memory pool	Transacciones que un nodo considera válidas pero que aún no están en ningún bloque. Público, distinto en cada nodo, y sin garantía de inclusión.
No repudio		Imposibilidad de negar haber firmado. Es una propiedad técnica sobre la clave, no una prueba jurídica sobre la persona.
Propagación por rumor	gossip	Cada nodo reenvía a sus vecinos lo que recibe y valida. No requiere servidor ni conocer la topología.
secp256k1		Curva elíptica de Bitcoin y Ethereum: $y^2 = x^3 + 7$ sobre un cuerpo finito, con generador y orden fijados por el estándar.
Serialización determinista		Representación en bytes que siempre produce la misma salida para los mismos datos. Condición necesaria para que la firma verifique en todos los nodos.



FUENTES

FUENTES PRIMARIAS

- Diffie, W. y Hellman, M. (1976). New Directions in Cryptography.
- Estándar SEC 2 — define secp256k1.
- RFC 6979 — uso determinista de DSA y ECDSA.
- EIP-55 — codificación de dirección con checksum en mayúsculas.

AMPLIACIÓN RECOMENDADA

- Antonopoulos, A. Mastering Bitcoin. Capítulo de claves y direcciones. Libre acceso.
- Antonopoulos, A. y Wood, G. Mastering Ethereum. Capítulo de criptografía. Libre acceso.
- Narayanan, A. et al. Bitcoin and Cryptocurrency Technologies. Capítulo 1, firmas digitales.



FIN DE LA SESIÓN 03

PRÓXIMA SESIÓN: CONSENSO Y EL TRILEMA

Cierra la Unidad I y trae el Quiz 1. Entregable de hoy: la mini-blockchain con las pruebas pasando, antes de que empiece.